Class 6 - **02 November 2026**

Build & Improving GenAI systems

Introduction to AI and Machine Learning

Executive Master in Artificial Intelligence for Business

Roberto Henriques · Fernando Peres

NOVA IMS INFORMATION MANAGEMENT SCHOOL

Table of Contents

01. What is a Harness?

02. The Mindset for Building and Continuously Improving GenAI Systems

03. Context

04. Tools and Integration

05. Workflows and Agents

06. Observability

07. Experiments

01. What is a **harness**?

LESSON

Harness overview

DEPTH

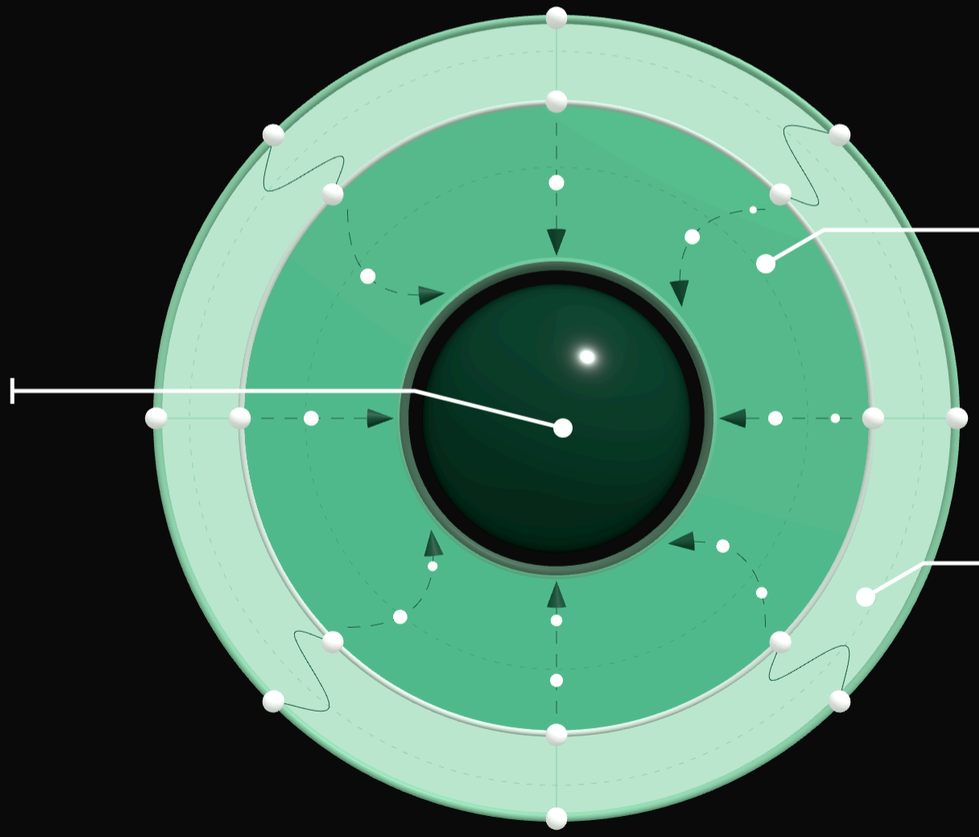
- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

Understand what a harness is: the layer around a GenAI model that contextualizes, extends, coordinates, constrains and operationalizes it. Tell apart the three layers of a GenAI system (the model, the harness, and the context and knowledge) and why the quality of the system depends on the harness as much as on the model.

Harness for GenAI systems capability extensions

A harness contextualizes, extends, coordinates, constrains, and operationalizes models to enable the development and execution of GenAI systems.*



The system that orchestrates and controls the intelligence.

HARNESS

MODEL

The intelligence that thinks and generates.

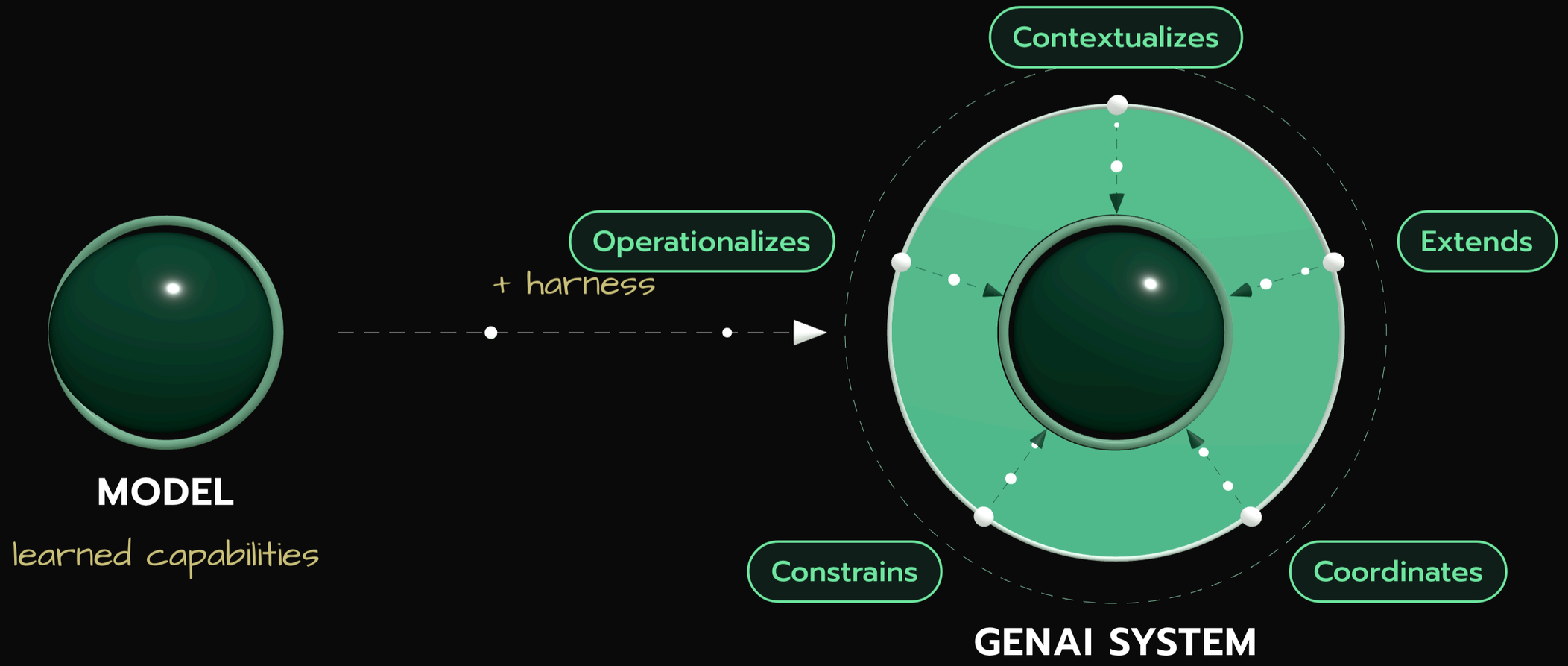
CONTEXT AND KNOWLEDGE

The information that grounds and guides the intelligence.

drag to rotate · double-click to reset

Model ≠ GenAI System

The model provides learned capabilities, while the harness contextualizes, extends, coordinates, constrains, and operationalizes them.



The components of a harness

A harness turns a model into a working system through five building blocks.

Context and knowledge integration	What the model sees: instructions, documents and retrieved data.
Memory and state management	What persists across steps, turns and sessions.
Tools and external capabilities	Actions beyond text: search, code execution and APIs.
Orchestration and control	Loops, workflows, routing, permissions and guardrails.
Evaluation mechanisms	Tests, metrics and feedback that check and improve outputs.

Three layers, three roles

1 · MODEL

Models are the intelligence

They provide the reasoning, understanding, and generation capabilities.

2 · HARNESS

The harness is the system around the intelligence

It orchestrates models, tools, context, and workflows to turn intelligence into reliable applications.

3 · CONTEXT & KNOWLEDGE

Context & knowledge provide what the intelligence needs

They ground the model in relevant data, memory, and knowledge to produce meaningful, context-aware responses.

Note: "intelligence" and "thinks" are shorthand. Models generate from learned patterns; they don't think the way people do. And context is more than knowledge: it also carries instructions, goals, and the current state of the task.

Implemented Harnesses — Ready-to-Use GenAI Systems

PRODUCT	PROVIDER	MAIN HARNESS CAPABILITIES
ChatGPT	OpenAI	Context management, memory, web search, tools, file processing, application integrations, agentic workflows
Claude	Anthropic	Conversational context, projects, artifacts, knowledge integration, tools, application connections
Gemini	Google	Multimodal interaction, tool use, information retrieval, Google ecosystem integrations
Microsoft Copilot	Microsoft	Microsoft 365 context, work and web grounding, document and email integration, agents
Perplexity	Perplexity AI	Real-time web search, source citation, retrieval, research workflows

Implemented Harnesses — Ready-to-Use GenAI Systems

PRODUCT	PROVIDER	MAIN HARNESS CAPABILITIES
OpenAI Codex	OpenAI	Coding-agent loop, repository context, terminal execution, testing, code editing, sandboxing, approvals
Claude Code	Anthropic	Agent loop, file-system and shell tools, project instructions, skills, hooks, MCP integrations, permissions
GitHub Copilot	GitHub/Microsoft	IDE context, repository integration, coding tools, agent workflows, code review
Cursor	Anysphere	Codebase indexing, file editing, terminal tools, agentic coding, workspace context
Replit	Replit	Cloud workspace, agent that plans and builds apps, code execution, databases, deployment
Lovable	Lovable	Prompt-to-app generation, full-stack scaffolding, visual editing, backend integrations, deployment

HARNESS · FRAMEWORKS

Harness Development Frameworks — Frameworks and SDKs for Building GenAI Systems

Implemented harnesses are products you use. These frameworks are the building blocks you use to make your own harness.

FRAMEWORK / SDK	ORGANIZATION	MAIN HARNESS COMPONENTS
LangChain	LangChain	Model integration, agents, tools, retrieval, middleware
LangGraph	LangChain	Stateful graphs, workflow orchestration, persistence, checkpoints, human-in-the-loop
CrewAI	CrewAI	Agents, roles, crews, tasks, flows, tools, delegation
OpenAI Agents SDK	OpenAI	Agent loop, tools, handoffs, guardrails, sessions, tracing
Microsoft AutoGen	Microsoft	Agent communication, multi-agent collaboration, message routing, orchestration
Semantic Kernel	Microsoft	Plugins, function calling, model connectors, process orchestration
LlamaIndex	LlamaIndex	Data connectors, indexing, RAG, retrieval, knowledge-centric agents

HARNESS · COMPARE

Implemented harnesses vs. harness development frameworks

One is a finished system ready to use; the other is a kit for building your own.

Dimension	Implemented harnesses	Harness development frameworks
Main purpose	Execute real user tasks	Build customized AI applications and agents
Target audience	End users, developers, organizations	AI engineers and software developers
Readiness	Operational system	Building blocks requiring configuration and implementation
Model integration	Already integrated	Developer-configured
Tools, memory, orchestration	Prebuilt or configurable	Supplied as primitives, abstractions, and APIs
Examples	ChatGPT, Claude Code, Codex	LangChain, LangGraph, CrewAI, OpenAI Agents SDK

02. The Mindset for Building and **Continuously Improving** GenAI Systems

Improving the Performance of GenAI Systems

Four complementary dimensions form a continuous improvement framework for GenAI systems, addressing their limitations and improving robustness, reliability, and output quality.

1 Model Optimization Techniques

- Pre-training (and continued pre-training)
- Post-training (RLHF and other post-training techniques)

Parametric memory



2 Harness and Capability Extension

- Memory systems and artifacts
- Context and knowledge integration
- Capabilities and tools
- Orchestration and control
- Execution and evaluation

3 Experiments and Validation

(offline)

- Benchmarks
- Datasets
- Test cases
- Metrics
- Evaluations

4 Observability and Continuous Improvement

- Telemetry
- Traces
- Metrics and evaluators
- User feedback

Four Dimensions of Continuous Improvement

1 Model Optimization Techniques

Improve model capabilities, alignment, efficiency, and task-specific performance through model selection, configuration, and adaptation.

2 Harness and Capability Extension

Extend and control model behavior through context, knowledge, memory, tools, orchestration, and runtime mechanisms.

3 Experimentation and Validation

Systematically test and compare models, prompts, contexts, tools, and system configurations to validate performance and guide design decisions.

4 Observability and Continuous Improvement

Monitor real-world behavior, identify limitations and failure modes, and use evidence to guide ongoing improvements.

An Interconnected Improvement System

Systemic perspective

Improving GenAI performance requires coordinated interventions across the model, harness, and system configuration.

Evidence-driven decisions

Experiments and production observability reveal what works, what fails, and what needs improvement.

Iterative evolution

Feedback loops connect evaluation, adaptation, and deployment, enabling GenAI systems to continuously improve over time.

KEY TAKEAWAY

Improving GenAI systems is not about optimizing the model alone. It is about continuously improving the entire system.

CONTINUOUS IMPROVEMENT

Why Model Optimization Alone Is Not Enough

Model optimization can improve capabilities, but it cannot address all the limitations of GenAI systems.

Cost and complexity	Fine-tuning and other optimization techniques can require substantial computational resources, specialized expertise, high-quality data, and ongoing maintenance.
Knowledge limitations	Model optimization alone does not provide reliable access to up-to-date, proprietary, or task-specific contextual information.
Memory limitations	It does not inherently provide persistent memory, long-term state management, or overcome context-window constraints.
Capability limitations	Models still need external tools to retrieve information, perform precise computations, interact with systems, and execute actions.
Reliability limitations	Optimization does not eliminate hallucinations, reasoning errors, or the need for runtime controls and validation.

KEY TAKEAWAY

Model optimization improves what a model can do; the harness extends what the system can access, remember, control, and execute.

Training models is expensive; improving the system is not

WHERE YOU IMPROVE	TYPICAL COST (A SENSE OF SCALE)	WHO DOES IT	TIME TO CHANGE
Pre-training	**$$$$$** $10M–$100M+ per run; e.g. Grok 4 ≈ $490M	Frontier labs	🕒🕒🕒🕒 Months; produces a new model
Post-training	**$$**$$$ ~$1M–$5M, depending on the technique. Frontier labs don't disclose; DeepSeek-R1's RL stage ≈ $0.3M	Labs, enterprises	🕒🕒🕒 Weeks; redone when the base model changes
Harness & context	**$**$$$$ Engineering time + pay-per-use inference	Product teams	🕒 Days; versioned and reversible
Experiments & observability	**$**$$$ Tooling + engineering time	Product teams	🕒 Continuous

Same models for everyone

Most companies use the same frontier models through an API. The advantage lies in the system built around them.

Cheap, fast, reversible

Harness changes ship in days, cost little, and carry over when a better model arrives.

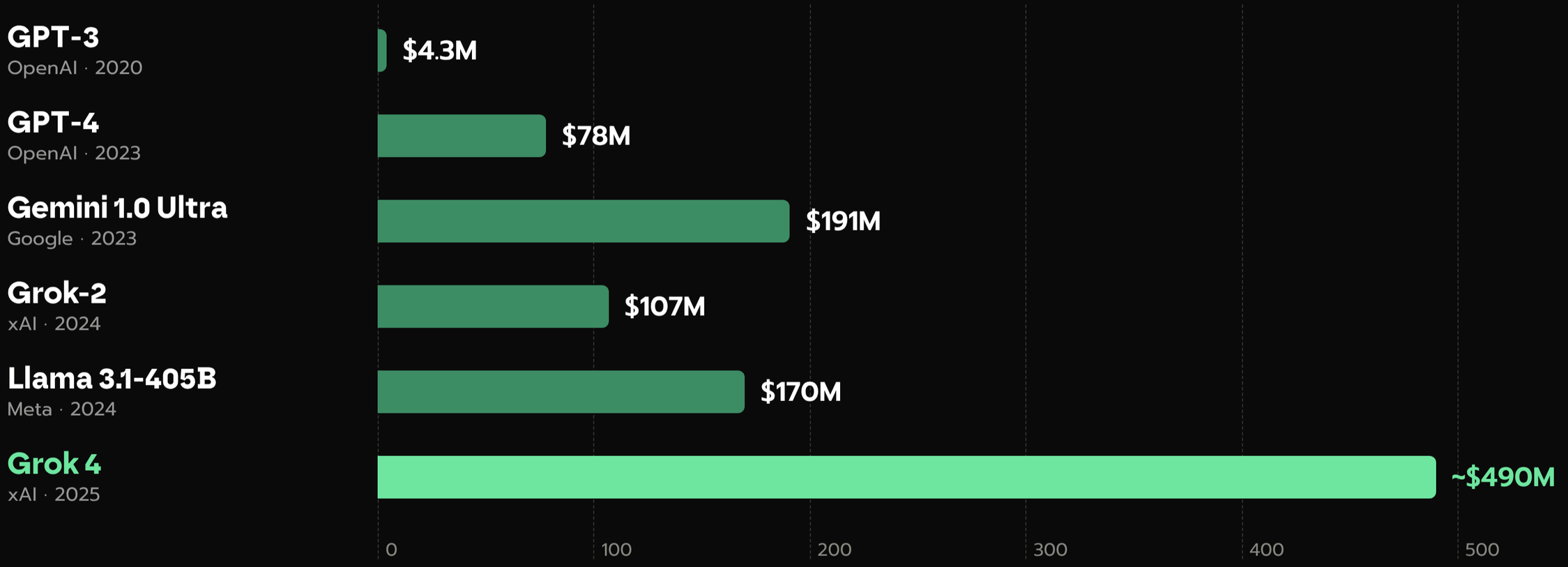
Evidence before spend

Experiments and observability show where the system fails, so effort goes where it pays off.

Indicative ranges, to give a sense of scale. Sources: Stanford HAI, AI Index 2024 (Epoch AI estimates); Epoch AI, Grok 4 (2025); DeepSeek-R1, *Nature* (2025).

The rising cost of training frontier models

Estimated compute cost of the final training run, in millions of US dollars.



Third-party estimates of compute for the final training run, priced at cloud-rental rates; they exclude research, staff, data and failed runs. Frontier training costs have grown ~2.4× per year since 2016 (95% CI 2.0–3.1×; Cottier et al., Epoch AI, 2024).
Sources: Stanford HAI, AI Index Report 2024 (GPT-3, GPT-4, Gemini 1.0 Ultra) and 2025 (Grok-2, Llama 3.1-405B), both using Epoch AI estimates; Epoch AI, *What did it take to train Grok 4?* (Sept 2025, median estimate).

Main takeaways

Training the model is rarely the answer

Pre-training costs tens to hundreds of millions; post-training needs data and expertise. And training alone can't fix knowledge cut-offs, memory, actions or hallucinations.

Grok 4 ≈ $490M per training run.
OpenAI: accuracy will never reach 100%

The harness can matter more than the model

Context, memory, tools, orchestration and guardrails expand and control what the model can do. The same model performs very differently in a different harness.

GPT-5 on Terminal-Bench 2.0: 33.9% → 49.6%, changing only the harness

Improve continuously, with evidence

Building and maintaining a GenAI system is a loop. Invest in experiments and observability, and let feedback, data and facts drive each change.

Read real traces → find failure modes → fix → measure again

Sources: Epoch AI, Grok 4 training resources (2025); OpenAI, *Why language models hallucinate* (2025); Terminal-Bench 2.0 leaderboard, as reported in *A Survey on Agent System and Harness Design* (arXiv, 2026); H. Husain & S. Shankar, AI evals guidance (2025–26).